Machine Learning-Aided Optimization of MOSFET Device Parameters

A PROJECT REPORT

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR AWARD OF THE DEGREE OF

Bachelor of Technology

by

ASHLY SKARIAH B230029EC

SHAUN JOSEPH B230705EC

V KETAN SARMA B231308EC

VIGNESH NARAYANAN B231325EC

under the supervision of

Dr. Jaikumar MG



DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

National Institute of Technology Calicut

NIT CAMPUS PO - 673601, KOZHIKODE, KERALA, INDIA

April 2026

© ASHLY SKARIAH, SHAUN JOSEPH, V KETAN SARMA, VIGNESH NARAYANAN, (2026)

NIT Calicut has the sole ownership of Patents and Software Copyrights resulting out of this Project work. NIT Calicut has the royalty-free permission to reproduce and distribute copies of this Project report for teaching and research as well as for dissemination of teaching and research in other academic institutions.

Acknowledgements

This project has been made possible due to the guidance and support of many people. We take this opportunity to express our sincere gratitude to our project guide, Dr. Jaikumar MG, Associate Professor, Department of Electronics and Communication Engineering, for his valuable guidance, encouragement, and continuous support throughout the course of this project.

We would also like to thank Dr. Sreelakha G, Professor and Head, Department of Electronics and Communication Engineering, for providing us with the opportunity, facilities, and institutional support necessary for the successful completion of this work.

We are grateful to the faculty members, research scholars, and staff of the Department of Electronics and Communication Engineering for their valuable suggestions, encouragement, and technical assistance.

We thank our family and friends for being a constant source of motivation, support, and inspiration. Above all, we thank the Almighty for giving us perseverance and strength throughout this journey.

Declaration

We hereby declare that except where specific reference is made to the work of others, the contents of this project report are original and have not been submitted in whole or in part for consideration for any other degree or qualification in this, or any other university. This project report is our own work and does not contain any outcome of work done in collaboration with others, except as specified in the text and Acknowledgements.



ASHLY SKARIAH B230029EC



SHAUN JOSEPH B230705EC



V KETAN SARMA B231308EC

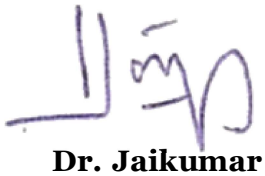


VIGNESH NARAYANAN B231325EC

NIT Calicut
Monday 20th April, 2026

Certificate

This is to certify that the project report entitled **Machine Learning-Aided Optimization of MOSFET Device Parameters** submitted by **ASHLY SKARIAH (B230029EC), SHAUN JOSEPH (B230705EC), V KETAN SARMA (B231308EC), and VIGNESH NARAYANAN (B231325EC)** to National Institute of Technology Calicut for the award of the degree of Bachelor of Technology is a bonafide record of the project work carried out by them under my supervision and guidance. The content of the project report, in full or parts, has not been submitted to any other institute or university for the award of any degree or diploma.



Dr. Jaikumar MG (Project Advisor)
Associate Professor
Dept. of Electronics and Communication Engineering
NIT Calicut

Dr. Sreelakha G
Professor and Head of Department
Dept. of Electronics and Communication Engineering
NIT Calicut

Place : NIT Calicut
Date : ___/___/_____

(Office Seal)

ABSTRACT

MOSFET scaling has been extended to the nanometer scale and that has made the simulation tools like Technology Computer Aided Design(TCAD) incompatible for this range of nanoscale level design . As they consume lots of resources and a single run might take several minutes to hours of wastage, this raises serious concerns for several alternatives due to the infeasibility of these simulation tools.

This report is a summary of the work done on ML alternative for TCAD simulations to predict the threshold voltage(Vth) using 3 critical parameters that have high correlation with the MOSFET's threshold. Namely we took Thickness of the oxide(Tox), channel implant dose(Nd) , and poly silicon gate length(L). A clean pre-processed data of 686 samples were generated using silvco TCAD simulations (Athena and Atlas) via Latin Hypercube Sampling.Four ML models are fitted over this generated data namely Polynomial regression, Random Forest , XG-Boost , and a artificial deep network of Neurons with tanh activation functions in hidden layers and reguralisation methods like dropout.

The best outcome of this experiment was XGBoost with a test accuracy of 0.96,outperforming the Neural Network's 0.95 accuracy on the same test data and Random forest of accuracy 0.904 on the same test data. Results have been clearly demonstrated using pair plots to have a clear correlation analysis , predicted vs actual scatter plots for each model and learing curves over epochs. The best performing model has been installed over an interactive web interface(https://website-demo-h607.onrender.com/) enables dynamic interaction with the user to otai the reuired threshold voltage on the user requirements.

Keywords: MOSFET, Threshold Voltage, TCAD, Machine Learning, XGBoost, Neural Networks, Surrogate Modeling, Digital Twin, Semiconductor Design Optimization.

CONTENTS

Acknowledgements

Declaration

Abstract **1**

List of Figures **5**

List of Tables **6**

1 Introduction **9**

1.1 Overview 9

1.2 Motivation 10

1.3 Problem Definition 10

1.4 Objectives 10

1.5 Contributions 11

2 Theoretical Background and Literature Survey **12**

2.1 MOSFET Threshold Voltage: Physics and Scaling Challenges 12

2.2 Failure of Linear Regression and the Necessity for Complex Models 12

2.3 TCAD-Enabled Machine Learning Frameworks 13

2.4 Model Calibration and Data Efficiency 14

2.5 Variability-Aware and Advanced Device Modeling 14

2.6 Theoretical Overview of Machine Learning Models and Their Relevance . 14

2.6.1 Polynomial Regression 14

2.6.2 Random Forest 15

2.6.3 XGBoost (Extreme Gradient Boosting) 15

2.6.4 Deep Neural Network (VthNet) 16

2.7 Positioning of This Work 17

3 Machine Learning-Aided MOSFET V_{th} Prediction **18**

3.1 Block Diagram 18

3.2 Methodology . . . 19
3.2.1 TCAD Device Simulation . . . 19
3.2.2 Dataset Overview . . . 20
3.2.3 Feature Engineering . . . 21
3.3 Algorithms . . . 21
3.3.1 Data Splits . . . 21
3.3.2 Model 1: Polynomial Regression (Baseline) . . . 21
3.3.3 Model 2: Random Forest Regressor . . . 22
3.3.4 Model 3: XGBoost Regressor . . . 22
3.3.5 Model 4: Deep Neural Network (VthNet) . . . 23
3.4 Techniques . . . 24
3.4.1 Latin Hypercube Sampling (LHS) . . . 24
3.4.2 Combining Dimensionality Reduction and Surrogate Modeling . . . 25
3.4.3 Web Deployment as a Digital Twin . . . 25

4 Results **26**
4.1 Experimental Setup . . . 26
4.1.1 Two-Parameter Baseline (Pre-Midsemester Work) . . . 26
4.2 Results . . . 27
4.2.1 Pair Plot Analysis . . . 27
4.2.2 Model Performance Comparison . . . 27
4.2.3 Predicted vs. Actual: Random Forest . . . 28
4.2.4 Neural Network Learning Curve . . . 29
4.2.5 Predicted vs. Actual: Neural Network . . . 29
4.2.6 Predicted vs. Actual: XGBoost . . . 30
4.2.7 Web Interface and Digital Twin Deployment . . . 30
4.3 Discussion . . . 31

5 Conclusion and Future Work **33**
BIBLIOGRAPHY . . . 34

A TCAD Simulation Scripts **36**
A.1 BUILDER: Silvaco Athena Process Script . . . 36
A.2 TESTER: Silvaco Atlas Device Script . . . 37

List of Figures

3.1 End-to-end process flow of the ML-assisted MOSFET optimization framework, from TCAD simulation through model training to web deployment. 18

4.1 Pair plot of the 3-parameter TCAD dataset ($n = 686$) showing pairwise scatter plots and univariate KDE distributions for all features and target. 27

4.2 Predicted vs. actual V_{th} scatter plot for Random Forest on the held-out test set ($R^2 = 0.9047$). It can be seen from the plot that while for lower V_{th} the model performs well, it struggles at higher V_{th} as Random Forest is not good at capturing smooth trends 28

4.3 Training and validation loss curves for VthNet over training epochs. The close gap between train and validation loss indicates good generalization without overfitting..... 29

4.4 Predicted vs. actual V_{th} scatter plot for the Neural Network (VthNet) on the held-out test set ($R^2 = 0.9546$). Points close to the line indicates accurate predictions. Deviations at high V_{th} (> 1.8 V) tell us about the sparsity of high-dose samples. 29

4.5 Predicted vs. actual V_{th} scatter plot for XGBoost on the held-out test set ($R^2 = 0.9666$). XGBoost achieves the tightest scatter of all models evaluated. This reflects the advantage of having sequential trees, where each tree corrects the residual error made by the previous one, thus acting as a correction over independent tree averaging (Random Forest) and smooth global function fitting (Neural Network). The characteristic tree-based discretization pattern is visible at extreme V_{th} values, representing the primary remaining limitation of this model class..... 30

4.6 The Digital Twin web interface showing the parameter input dashboard (left) and the predicted V_{th} output display (right). 31

List of Tables

3.1 TCAD Simulation Parameter Ranges..... 20
3.2 XGBoost Hyperparameters.....22
3.3 VthNet Architecture.....23

4.1 Two-Parameter Model Performance (Pre-Midsemester) 26
4.2 3-Parameter Model Performance Comparison (Post-Midsemester)..... 28

List of Abbreviations

ATLAS	Silvaco Device Simulation Tool
ATHENA	Silvaco Process Simulation Tool
BPTT	Backpropagation Through Time
DIBL	Drain-Induced Barrier Lowering
DFM	Design for Manufacturability
FCNN	Fully Connected Neural Network
LHS	Latin Hypercube Sampling
MAE	Mean Absolute Error
ML	Machine Learning
MOSFET	Metal-Oxide-Semiconductor Field-Effect Transistor
MSE	Mean Squared Error
NLP	Natural Language Processing
NN	Neural Network
PR	Polynomial Regression
PTQ	Post-Training Quantization
RF	Random Forest
RMSE	Root Mean Squared Error
SCE	Short Channel Effects
TCAD	Technology Computer-Aided Design
XGB	XGBoost (Extreme Gradient Boosting)

List of Symbols

V_{th}	Threshold voltage
V_{fb}	Flat-band voltage
ϕ_F	Fermi potential
Q_{dep}	Depletion charge density
Q_f	Interface charge density
C_{ox}	Gate oxide capacitance per unit area
ε_{ox}	Permittivity of gate oxide
ε_{si}	Permittivity of silicon
t_{ox}	Gate oxide thickness
T_{ox}	Gate oxide growth time (ML parameter)
N_d	Channel implant dose
L	Effective gate length
p_{1x}	Gate etch position
V_{GS}	Gate-to-source voltage
V_{DS}	Drain-to-source voltage
I_D	Drain current
g_m	Transconductance ($\partial I_D/\partial V_{GS}$)
R^2	Coefficient of determination
$\hat{V}_{th}$	Predicted threshold voltage
$f(\cdot)$	Learned model function
$\phi(\cdot)$	Polynomial feature map
w_i	Model weights / coefficients
T	Number of trees in ensemble
G, H	First and second gradient sums (XGBoost)
Ω	Regularization term
γ	Minimum loss reduction for split
λ	L2 regularization coefficient
$\sigma(\cdot)$	Activation function
$W^{(l)}, b^{(l)}$	Weight matrix and bias of layer l

CHAPTER 1

Introduction

1.1 Overview

Moore's Law has historically governed the relentless scale down of semiconductor devices, which has pushed MOSFET dimensions into the deep sub-micron and nanometer range. When channel lengths get shorter than 100 nm, classical long-channel models based on idealized electrostatics become less accurate. Short channel effects such as DIBL (drain-induced barrier lowering), velocity saturation, hot-carrier injection, and threshold voltage roll-off are all examples of phenomena that become more relevant. To get an accurate picture of a device, the need to use physics-based simulation becomes prominent. [3].

Silvaco Athena (process simulation) and Atlas (device simulation) are two examples of Technology Computer-Aided Design (TCAD) tools that solve the full set of coupled drift-diffusion, Poisson, and continuity equations for carriers in a defined device geometry. TCAD gives results that are physically sound, but each simulation can take anywhere from tens of minutes to hours, which makes it too expensive to use for design space exploration with hundreds or thousands of parameter combinations.

Machine learning (ML) offers a compelling solution: given a sufficiently representative dataset of TCAD outputs, a trained ML model can make predictions in milliseconds, enabling rapid sweeps across parameter spaces and facilitating inverse design—i.e., finding the fabrication parameters that yield a desired device characteristic. This project builds upon earlier two-parameter regression work (pre-midsemester) and substantially extends it by: (i) incorporating a third Parameter(gate etch position/length), (ii) implementing and comparing ensemble and deep learning models (XGBoost and Neural Networks), and (iii) deploying a web-based digital twin for interactive design exploration.

The rest of this paper is set up like this: Section II looks at other research on how to combine TCAD and ML. Section III makes clear what the problem is. Section IV talks about the whole process, including TCAD scripts, making datasets, feature engineering, and model architectures. Section V shows and talks about the results. Section VI talks about the web deployment. Section VII ends with instructions for what to do next. In

the Appendix, you can find the full TCAD simulation scripts.

1.2 Motivation

MOSFET scaling has been extended to the nanometer scale and that has made the simulation tools like Technology Computer Aided Design (TCAD) incompatible for this range of nanoscale level design. As they consume lots of resources and a single run might take several minutes to hours of wastage, this raises serious concerns for several alternatives due to the infeasibility of these simulation tools.

Machine learning (ML) offers a compelling surrogate modeling solution: given a sufficiently representative dataset of TCAD outputs, a trained ML model can make predictions in milliseconds, enabling rapid sweeps across parameter spaces and facilitating inverse design. The need for accurate, fast, and deployable surrogate models to replace expensive TCAD simulations motivates the entire scope of this project.

1.3 Problem Definition

The threshold voltage V_{th} of a MOSFET is a critical parameter that determines the switching behavior of the transistor. Predicting V_{th} from process parameters is a non-trivial regression problem because the underlying physics involves non-linear electrostatic coupling, feature interactions, and short-channel effects that classical linear models cannot capture.

Formally, the problem is to learn a surrogate function $f : R^3 \rightarrow R$ such that:

$$\hat{V}_{th} = f(T_{ox}, N_d, L) \tag{1.1}$$

where T_{ox} is the gate oxide growth time, N_d is the channel implant dose, and L is the effective gate length (controlled by gate etch position p_{1x}). The function f is trained on TCAD-generated data and must generalize accurately to unseen parameter combinations within the design space.

1.4 Objectives

The objectives of this project are formally stated as:

1. **Data Generation:** Use Silvaco TCAD (Athena + Atlas) to make a structured dataset of MOSFET V_{th} values by changing three process parameters in a systematic

way: gate oxide growth time (T_{ox}), channel boron implant dose (N_d), and gate length (L).

2. **Surrogate Modeling:** Train and compare several machine learning models, such as polynomial regression, XGBoost, and a deep neural network, to predict V_{th} from the three process inputs. The models should have a R^2 value of more than 0.95 on test data that was not used for training.

3. **Model Explainability:** To find out how each parameter affects V_{th} and to look for any non-linearities or interactions, use pair plots, predicted-vs-actual scatter plots, and learning curves to look at the relationships between features.

4. **Deployment:** Deploy the best-performing model in a web-based digital twin interface enabling forward prediction (parameter inputs → V_{th} output) and interactive design space visualization.

1.5 Contributions

The main contributions of this work, along with a summary of results, are:

- A 686-sample TCAD dataset created by diligently altering gate oxide time, channel implant dose, and gate etch position utilizing Silvaco Athena/Atlas via Latin Hypercube Sampling.

- Five models were put into action and compared: linear regression, polynomial regression (degree 2), random forest, XGBoost, and a custom neural network called VthNet.

- **XGBoost achieved the best overall performance** ($R^2 = 0.9666$), outperforming the Neural Network VthNet ($R^2 = 0.9546$) by $\Delta R^2 \approx 0.012$ and Random Forest ($R^2 = 0.9047$) by $\Delta R^2 \approx 0.062$. This is in line with what is commonly observed in problems with tabular data, where gradient-boosted trees tend to do better because they can focus on harder samples by correcting mistakes one at a time.

- Setting up an interactive web-based digital twin for real-time forward prediction and basic design exploration, accessible at https://website-demo-h607.onrender.com/.

CHAPTER 2

Theoretical Background and Literature Survey

2.1 MOSFET Threshold Voltage: Physics and Scaling Challenges

The threshold voltage V_{th} of an n-channel MOSFET can be expressed analytically as:

$$V_{th} = V_{fb} + 2\phi_F + \frac{Q_{dep}}{C_{ox}} \tag{2.1}$$

The above equation includes flat-band voltage, the Fermi potential, the depletion charge density, and the gate oxide capacitance per unit area. This equation shows that the oxide thickness (t_{ox}) and the channel doping (N_d through Q_{dep} and ϕ_F) are the most significant factors. For short-channel devices, extra second-order terms take into account charge sharing between the gate and the depletion regions of the source and drain. This creates a gate-length dependence that can't be captured analytically without making a lot of assumptions.[2].

2.2 Failure of Linear Regression and the Necessity for Complex Models

Linear regression assumes that the target variable is a weighted sum of the input features without any interaction effects.

$$\hat{V}_{th} = w_0 + w_1 T_{ox} + w_2 N_d + w_3 L \tag{2.2}$$

This model fundamentally fails to capture the physics of threshold voltage for several reasons.

Non-linear electrostatic coupling: The depletion charge density $Q_{dep} = \sqrt{4\varepsilon_{si} q N_d \phi_F}$ depends on N_d through a square root, and the Fermi potential $\phi_F = (kT/q)\ln(N_d/n_i)$

depends on N_d through a logarithm. The oxide capacitance $C_{ox} \propto 1/t_{ox}$ creates a hyperbolic relationship with the thickness of the oxide. Generally, A linear model would give N_d a constant slope across the whole doping range, however none of these things are captured.

Feature interactions: The joint effect of T_{ox} and N_d on V_{th} is multiplicative, not additive. A thick oxide combined with high doping produces a disproportionately large V_{th} increase, since both the reduced C_{ox} and the increased Q_{dep} act simultaneously. Linear models, by construction, cannot represent this interaction term $T_{ox} \cdot N_d$.

Short-channel charge sharing: For small gate lengths L, the depletion region beneath the channel is shared between the gate, source, and drain electrodes. This sharing reduces the effective Q_{dep} controlled by the gate, causing V_{th} roll-off—a strongly non-linear and non-monotonic effect in L that linear regression maps incorrectly to a constant coefficient.

Empirically, this limitation is evident in our results: linear regression achieved only $R^2 = 0.887$ on the three-parameter dataset, confirming that a significant portion of V_{th} variance is attributable to non-linear and interaction terms that linear models are fundamentally incapable of representing.

Why complexity is necessary: Semiconductor device characteristics occupy a multi-dimensional, smoothly curved manifold in the process parameter space. Capturing this manifold requires models with either (a) explicit non-linear basis functions (polynomial regression), (b) adaptive piecewise approximation (tree ensembles), or (c) learned hierarchical representations (deep neural networks). As the number of parameters and the range of their interactions grows—particularly when extending toward seven-parameter optimization—the complexity gap between linear and non-linear models widens substantially. The following subsections review each class of advanced model and its specific relevance to this problem.

2.3 TCAD-Enabled Machine Learning Frameworks

The TCAD Simulation and machine learning are in rapid exponential growth[1] demonstrates that TCAD datasets improve the reliability of the ML models rapidly especially for feedback field effect transistors by combining physics trends into tarining data . The key insight is that TCAD data is noiseless and can be generated in a controlled, structured manner—making it ideal for supervised learning. Jungmann et al. [4] gives a layout to generate the training data and focuses on controlled parameter sampling (Latin Hypercube Sampling) and physics based validating model for the reduction of uncertanity

2.4 Model Calibration and Data Efficiency

Wang et al. [3] utilized neural networks and ensemble classifiers for TCAD model calibration, resulting in substantial decreases in parameter fitting duration. A major improvement was a pre-filtering system that got rid of parameter combinations that didn't converge. This made data generation more efficient, going from 32% to almost 79%. This solves a very important real-world problem: not all TCAD runs converge, especially when the parameters are very high or low, which makes numerical stability less stable. In our study, 14 out of 700 generated simulation points lacked V_{th} values, necessitating data cleaning prior to model training.

Choi et al. [2] conducted a systematic evaluation of regression algorithms for automatic V_{th} prediction, illustrating that ensemble methods (random forest, gradient boosting) consistently surpass simple linear or polynomial models in the presence of intricate feature interactions. This prompted our examination of XGBoost in conjunction with deep neural networks.

2.5 Variability-Aware and Advanced Device Modeling

As device dimensions reduces, variations in parameters caused by the process have a greater effect on performance and yield. Lyu et al. [5] illustrated ML-based variability-aware modeling for nanosheet FETs, wherein statistical distributions of process parameters are transmitted through a trained surrogate to yield output distributions, a vital function for design for manufacturability (DFM). Ji et al. [6] enhanced the robustness of GaN transistors through stacked regression that integrates XGBoost and fully connected neural networks (FCNN), demonstrating that ensemble methods surpass individual learners in sparse-data contexts. Zhang et al. [7] utilized gradient-boosting and neural network techniques for FinFET modeling, indicating that deep architectures with regularization (dropout, batch normalization) are crucial for small datasets (<1000 training samples) to avoid overfitting.

2.6 Theoretical Overview of Machine Learning Models and Their Relevance

2.6.1 Polynomial Regression

Polynomial regression is linear in parameter but has the features combined and recombined in higher degree higher than 1.This modle calculates the following degree 2 expan-

sion over these men $\{T_{ox}, N_d, L\}$:

$$\hat{V}_{th} = \sum_{i} w_i x_i + \sum_{i \leq j} w_{ij} x_i x_j \tag{2.3}$$

We get 10 basis functions , one of the terms which is bias. The term $w_{12} T_{ox} N_d$ represents the electrostatic effect. Polynomial regression is an easy way to implement and the power of computation is far low than expected, but it has the issue called "curse of dimensionality reduction " when you add degree-d to p features, you get $\binom{p+d}{d}$ terms, which grow quickly as p and d grow. More importantly, polynomials are global functions.Polynomial is a universal function it is a high degree polynomial which fits well for densely populated areas of the training data but highly oscillates in sparse areas of data thus making them not good option for corners of the 3d space. Here a lower order 2 degree polynomial model explains well the V_{th}'s variance with respect to the 3 MOSFET parameters chosen.

2.6.2 Random Forest

Random Forest is made up of T decision trees, each of which is trained on a bootstrap sample of the data and a random set of features at each split. The prediction from the ensemble is:

$$\hat{V}_{th} = \frac{1}{T} \sum_{t=1} f_t(\mathbf{x}) \tag{2.4}$$

where f_t is the t-th tree. Averaging over trees that were trained separately lowers variance without raising bias too much (the "bias-variance tradeoff"). At each split, the feature randomization (choosing $\lfloor \sqrt{p} \rfloor$ features for classification or $p/3$ for regression) makes the trees less correlated, which makes the ensemble more diverse. When it comes to predicting V_{th}, Random Forest naturally deals with the multiplicative interactions between T_{ox}, N_d, and L. Each tree divides the parameter space into rectangular areas and makes a constant prediction in each area. The collection of many partitions gives a piecewise-constant approximation of the smooth V_{th}. The main problem is that predictions for individual trees looks like a step function, so they don't work well for smooth curves. This is the case for our V_{th} prediction, which is also continuous and smooth.

2.6.3 XGBoost (Extreme Gradient Boosting)

XGBoost is a regularized gradient boosted tree algorithm that builds an additive ensemble, one tree at a time. Each new tree f_k is trained to reduce the residual error of all the

other trees that came before it. At iteration k, the objective function is:

$$\mathrm{L}^{(k)} = \sum_{i=1}^{n} \ell\left(y_i, \hat{y}_i^{(k-1)} + f_k(\mathbf{x}_i)\right) + \Omega(f_k) \tag{2.5}$$

where ℓ is the mean squared error loss and $\Omega(f_k) = \gamma T + \frac{1}{2}\lambda \|\mathbf{w}\|^2$ is a regularization term that penalize tree complexity (the number of leaves T) and the magnitudes of the leaf weights. XGBoost uses a second-order Taylor approximation of the loss to find the best way to split the data:

$$\text{Gain} = \frac{1}{2}\left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda}\right] - \gamma \tag{2.6}$$

where $G = \sum_i g_i$ and $H = \sum_i h_i$ are the first and second derivative sums for samples in a node. This second-order optimization makes XGBoost much better at using data than regular gradient boosting. In our case, the sequential correction mechanism lets XGBoost make better predictions in areas where earlier trees don't do well, especially at very high doping levels where V_{th} behaves the least linearly.

2.6.4 Deep Neural Network (VthNet)

A neural network represents V_{th} as a combination of acquired non-linear transformations:

$$\hat{V}_{th} = f^{(L)} \circ f^{(L-1)} \circ \cdots \circ f^{(1)}(\mathbf{x}) \tag{2.7}$$

where each layer $f^{(l)}(\mathbf{h}) = \sigma(\mathbf{W}^{(l)}\mathbf{h} + \mathbf{b}^{(l)})$ applies a learnable affine transformation followed by a non-linear activation σ. The universal approximation theorem says that a single hidden layer network, that is wide enough, can get as close as you want to any continuous function on a compact domain. The practical benefit of depth is that it enables hierarchical feature composition. Shallow layers learn low-level feature combinations, such as how T_{ox} and N_d together affect oxide field strength. Deeper layers then put these together into representations that are consistent with higher-level physics.

The Tanh activation function is what we used for our physics-based regression task. The function

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \tag{2.8}$$

is especially useful because it is smooth (infinitely differentiable), stays between -1 and 1, and its sigmoidal shape naturally fits the saturation behavior seen in V_{th} vs. N_d curves at high doping levels. ReLU activations, on the other hand, would cause piecewise-linear responses that doesn't match the smooth physical manifold. The bottleneck architecture

(which compresses to three neurons before the output) acts as a dimensionality regularizer, making the network encode V_{th} using only three internal variables, which is in line with the three physical degrees of freedom in the problem. Using dropout (0.3) and the Adam optimizer with weight decay avoids overfitting on the small 686-sample dataset.

2.7 Positioning of This Work

Our project further contributes to this body of knowledge in three ways:

1. We work in the less explored three-dimensional process parameter space (oxide time, channel dose, gate etch)
2. We directly compare polynomial regression, Random Forest, XGBoost, and a custom deep NN architecture using the same dataset.
3. We connect simulation and deployment through a full-stack web application that acts as a digital twin for real-time design exploration.

CHAPTER 3

Machine Learning-Aided MOSFET V_{th} Prediction

3.1 Block Diagram

The project workflow proceeds through four major stages, as illustrated in Figure 3.1: (1) process simulation in Silvaco Athena to generate device structures, (2) device simulation in Silvaco Atlas to extract I_D–V_G characteristics and V_{th}, (3) machine learning model training and comparison, and (4) deployment of the best model via a web interface.

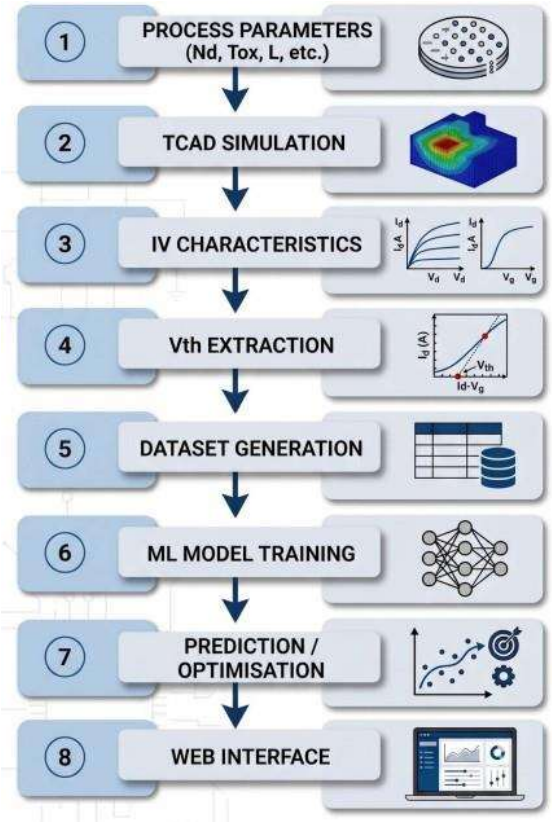


Figure 3.1: End-to-end process flow of the ML-assisted MOSFET optimization framework, from TCAD simulation through model training to web deployment.

3.2 Methodology

3.2.1 TCAD Device Simulation

Process Simulation (Athena – BUILDER Module)

The MOSFET structure is fabricated in simulation using Silvaco Athena with the following steps:

1. **Mesh Definition:** A 2D simulation mesh is defined over a $0.6\ \mu m \times 0.8\ \mu m$ domain with fine vertical spacing ($0.01\ \mu m$) at the surface to resolve the oxide-silicon interface.
2. **Substrate Initialization:** A p-type silicon substrate is initialized with phosphorus background doping of $10^{14}\ cm^{-3}$.
3. **Field Oxide Growth:** Dry oxidation at 1000°C for 30 minutes to form isolation oxide, followed by etching.
4. **Well Formation:** Boron implant at dose $8 \times 10^{12}\ cm^{-2}$, 100 keV, followed by wet oxidation drive-in at 950°C.
5. **Gate Oxide Growth (ML Knob 1 – T_{ox}):** The ML-controlled gate oxide growth step. For the example point shown in the Appendix, oxidation time is 24.45 minutes at 925°C in dry oxygen.
6. **Channel Implant (ML Knob 2 – N_d):** Boron threshold adjust implant. For the example, dose = $2.44 \times 10^{11}\ cm^{-2}$ at 10 keV.
7. **Polysilicon Gate Deposition:** $0.4\ \mu m$ poly gate layer deposited.
8. **Gate Patterning (ML Knob 3 – L):** Left etch of polysilicon at position $p_{1x} = 0.403\ \mu m$, defining the effective gate length.
9. **Source/Drain Formation:** Phosphorus halo implant, spacer oxide, arsenic source/drain implant ($5 \times 10^{15}\ cm^{-2}$, 50 keV), and anneal at 900°C.
10. **Metallization:** Aluminum contacts deposited and patterned; structure mirrored to define full device. Electrodes placed at gate (x=0.5, y=0.1), source (x=0.1), drain (x=1.1), and substrate (backside).

The three ML knobs—oxide time, channel dose, and gate etch position—are the only parameters varied between simulation runs; all other steps remain identical.

Device Simulation (Atlas – TESTER Module)

The saved device structure (my_device.str) is loaded into Silvaco Atlas for electrical characterization:

- Physical models: cvt (Lombardi mobility model for surface scattering) and srh (Shockley-Read-Hall recombination).
- Gate contact is defined as n-type poly for the purpose of assigning work function.
- At the Si-SiO_2 interface, the charge density is $Q_f = 3 \times 10^{10}$ cm^{-2}.
- Newton solver for coupled Poisson-continuity equations.
- $V_{DS} = 0.1$ V was applied, and V_{GS} was varied from 0 to 3.0 V with 50 mV increments.
- V_{th} was calculated by applying the maximum transconductance (linear extrapolation) method:

$$V_{th} = V_{GS}\,_{\max(g_m)} - \frac{V_{DS}}{2} \tag{3.1}$$

where $g_m = \partial I_D / \partial V_{GS}$ is evaluated numerically from the simulated I_D–V_G curve.

3.2.2 Dataset Overview

A total of 700 TCAD simulation points were generated through Latin Hypercube Sampling (LHS) over the three-dimensional parameter space:

Table 3.1: TCAD Simulation Parameter Ranges

Parameter	Symbol	Range
Gate oxide growth time	T_{ox}	1–25 min
Channel implant dose	N_d	10^{11}–5×10^{12} cm^{-2}
Gate etch position	p_{1x}	0.25–0.50 μm
Target	V_{th}	−0.02–2.85 V

Of the 700 runs, 14 did not converge (non-convergent Atlas solutions, resulting in missing V_{th} values). After dropping these rows, the clean dataset contains **686 samples**. The wide V_{th} range reflects the strong dependence on channel doping: lightly doped channels yield near-depletion-mode behavior, while heavily doped channels exhibit high enhancement-mode thresholds.

3.2.3 Feature Engineering

The channel dose N_d covers approximately 1.5 decades from 10^{11} to 5×10^{12} cm^{-2}. Since V_{th} is dependent on $\sqrt{N_d}$ (through $Q_{dep} = \sqrt{4\varepsilon_{si} q N_d \phi_F}$), a $\log_{10}$ transformation was applied to the dose before NN training:

$$\tilde{N}_d = \log_{10}(N_d) \tag{3.2}$$

This change keeps the dose-V_{th} relationship linear. We used the raw dose for XGBoost because tree-based models don't change when you change the order of the features.

3.3 Algorithms

3.3.1 Data Splits

Using stratified random splitting with `random_state=42` for reproducibility, the dataset was split into three parts: training (70%), validation (15%), and test (15%). The validation set was used to adjust hyperparameters and stop early; the test set was not used at all until the final evaluation.

3.3.2 Model 1: Polynomial Regression (Baseline)

Relevance and implementation: A scikit-learn `Pipeline` combining `PolynomialFeatures` (degree=2) with `LinearRegression` was implemented as the analytical baseline. This model constructs all pairwise interaction and squared terms from the three input features:

$$\begin{aligned}\phi(\mathbf{x}) = [1,\ & T_{ox},\ N_d,\ L,\ T_{ox}^2,\ T_{ox}N_d,\ T_{ox}L,\\ & N_d^2,\ N_dL,\ L^2]\end{aligned} \tag{3.3}$$

yielding 10 polynomial basis functions. The cross-term $w_{12}T_{ox}N_d$ encodes the joint electrostatic effect of oxide capacitance and depletion charge. No regularization was applied. While this model captures first-order interactions, it cannot represent the $\sqrt{N_d}$ electrostatic dependence or the gate-length-dependent charge-sharing roll-off, making it a natural ceiling-benchmark for non-regularized analytical approaches.

Key limitation in context: The polynomial basis is fixed at training time and cannot adapt to the discovery of new interaction structures. As we scale toward more parameters (e.g., 7-parameter optimization), the number of degree-2 basis functions grows as $\binom{p+2}{2}$, reaching 36 for $p = 7$, and degree-3 expansion explodes to 120—making the basis impractical and over-parameterized relative to the training set.

3.3.3 Model 2: Random Forest Regressor

Relevance and implementation: Random Forest was implemented using scikit-learn's `RandomForestRegressor` with 500 trees and maximum depth 10. The ensemble constructs each tree on a bootstrap sample and selects $\lfloor p/3 \rfloor = 1$ random feature at each split, forcing different trees to specialize in different parameter subspaces. The averaging mechanism:

$$\hat{V}_{th} = \frac{1}{500} \sum_{t=1}^{500} f_t(T_{ox}, N_a, L) \tag{3.4}$$

reduces prediction variance without introducing systematic bias from any single tree's partition boundaries.

Speciality in solving the problem: Random Forest handles the multi-scale nature of the feature space (dose spanning four orders of magnitude, gate etch in micrometers, time in minutes) without requiring feature normalization, since decision trees partition based on rank-order comparisons. Feature importance scores can be computed from the mean impurity decrease at each split, providing insight into which process knob dominates V_{th} variability. However, Random Forest's piecewise-constant response surface introduces discretization artifacts at high V_{th} values, as seen in the predicted vs. actual scatter plot (Fig. 4.2).

3.3.4 Model 3: XGBoost Regressor

Relevance and implementation: XGBoost was implemented with the hyperparameters given below:

Table 3.2: XGBoost Hyperparameters

Hyperparameter	Value
n_estimators	300
learning_rate	0.05
max_depth	5
subsample	0.8
colsample_bytree	0.8
random_state	42

Speciality in solving the problem: The mechanism of sequential boosting of XGBoost emphasizes on the samples that are not easy to predict.This effect can be seen in the areas of high doping , thick oxide area of the parameter space where Vth varies at

high pace. The amount of tress required to reach accuracy as same as that of the second order gradient optimization cpmpared to standard gradientr boosting.The regularization terms (γ on leaf count and λ on leaf weights) hold the model from remembering the data of the training set of 686 samples, the subsapling parameters enable the tress to have random variations among them, with a R^2 value of 0.9666. This is better than the Neural Network ($\Delta R^2 \approx +0.012$) and the Random Forest ($\Delta R^2 \approx +0.062$). The XGBoost has a mechanism of sequential error detection and correction that has a significant edge over other models.It makes good predictions on samples that are sparse and hard to predict at corners of doping and oxide thickness corners better than the other models experimened in this project.

3.3.5 Model 4: Deep Neural Network (VthNet)

Relevance and implementation: We made a custom PyTorch neural network architecture (named `VthNet`) designed specifically for this regression task:

Table 3.3: VthNet Architecture

Layer	Output Dim	Activation
Input	3	–
Linear(3 → 64)	64	Tanh
Linear(64 → 64)	64	Tanh
Dropout(0.3)	64	–
Linear(64 → 64)	64	Tanh
Dropout(0.3)	64	–
Linear(64 → 3)	3	Tanh
Linear(3 → 1)	1	(linear)

Design rationale:

- *Tanh activations:* Tanh is better than ReLU because it is smooth (differentiable everywhere), bounded, and its S-shaped response naturally mimics the monotonic-with-saturation behavior of V_{th} vs. process parameters. ReLU's piecewise-linear response can make learned functions behave like they have false bumps.
- *Bottleneck layer (64 → 3):* The compression to three neurons before the final output layer creates an information bottleneck that forces the network to learn the simplest way to represent the physics, which is like the three physical degrees of freedom in the problem.

- *Dropout (0.3):* Applied after the second and third hidden layers to stop neurons from adapting to each other, which is important because the dataset is small (686 samples).
- *Weight decay (10^{-4}):* L2 regularization on optimizer weights provides additional overfitting control.

Speciality in solving the problem: The neural network learns a globally smooth, continuously differentiable function, which is different from tree-based models. This property directly matches the physical expectation that V_{th} changes smoothly with process parameters. The gradient-based optimization lets VthNet find mixed interaction terms of any shape, not only those that fit a certain polynomial basis. This is the most scalable architecture because the number of parameters can go up to 7. Even with these benefits, VthNet only gets $R^2 = 0.9546$ on the test set, which puts it in second place overall behind XGBoost. This is consistent with the widely accepted fact that gradient-boosted trees tend to do better than neural networks on small, structured tabular datasets.

Training procedure:

- Optimizer: Adam with $lr = 10^{-3}$ and weight decay 10^{-4}.
- Loss: Mean Squared Error (MSE).
- Scheduler: `ReduceLROnPlateau` with a factor of 0.5 and a patience of 15 epochs.
- Early stopping: Training stops if the validation loss doesn't get better for 50 epochs in a row, and the best weights are restored.
- Batch size: 32; maximum epochs: 1000.
- Before training, the input features were normalized with `StandardScaler`.

3.4 Techniques

3.4.1 Latin Hypercube Sampling (LHS)

Latin Hypercube Sampling ensures that the 700 simulation points are spread uniformly across the entire three-dimensional parameter space. Unlike random sampling, LHS partitions each parameter axis into N equal-probability intervals and places exactly one sample in each interval per dimension, ensuring coverage of all regions including corners and edges of the design space. This is critical for training ML models that generalize across the full parameter range.

3.4.2 Combining Dimensionality Reduction and Surrogate Modeling

The bottleneck layer in VthNet (compressing from 64 to 3 neurons before the output) functions as an implicit dimensionality reduction step. This architectural choice forces the network to encode threshold voltage prediction through only three latent variables, mirroring the three physical process parameters. Combined with dropout regularization, this technique improves generalization on the small 686-sample dataset.

3.4.3 Web Deployment as a Digital Twin

The trained XGBoost model is deployed as a digital twin—a real-time software counterpart of the physical MOSFET fabrication process. Users interact with the twin through sliders for T_{ox}, N_d, and L, obtaining predicted V_{th} values in milliseconds without running a single TCAD simulation. The Spring Boot backend handles RESTful inference requests, and the full stack is hosted on the Render cloud platform, accessible at https://website-demo-h607.onrender.com.

CHAPTER 4

Results

4.1 Experimental Setup

All ML models were trained and evaluated on the same 686-sample TCAD dataset. The dataset was split into training (70%), validation (15%), and test (15%) subsets using stratified random splitting with `random_state=42`. Polynomial regression and Random Forest were implemented in scikit-learn. XGBoost was implemented using the `xgboost` Python library. VthNet was implemented in PyTorch and trained on CPU with the Adam optimizer. All experiments were run on a standard laptop without GPU acceleration.

4.1.1 Two-Parameter Baseline (Pre-Midsemester Work)

The initial project phase used a two-parameter dataset which comprised of only 2 parameters implant dose and oxide thickness, with fixed $L = 0.35\ \mu$m, containing 500 clean TCAD simulation points. Results are summarized in Table 4.1.

Table 4.1: Two-Parameter Model Performance (Pre-Midsemester)

Model	R^2	MSE	MAE	Observation
Linear Regression	0.9655	0.0122	0.0034	Strong linear fit
Polynomial (deg. 2)	0.9910	–	–	Captures non-linear electrostatics

The polynomial model's ($R^2 = 0.991$) results look better than that of linear regression, but they were observed to overfit on the data, as the 2 parameters taken were highly correlated and tend to follow a linear relationship.

4.2 Results

4.2.1 Pair Plot Analysis

The pair plot computed over the full 3-parameter dataset (Figure 4.1) shows a few important relationships:

- N_d **vs.** V_{th}**:** A strong positive, almost monotonic correlation is observed. As the channel doping increases, Q_{dep} also increases, so a higher gate voltage is needed to invert the channel. The trend is slightly concave (sub-linear), which aligns with the $\sqrt{N_d}$ relationship. This is the most influential predictor.
- T_{ox} **vs.** V_{th}**:** A moderate positive correlation is seen. Longer oxidation times lead to thicker oxides, which reduces $C_{ox} = \varepsilon_{ox}/t_{ox}$. As a result, a higher V_{GS} is required to achieve the same surface potential.
- **Gate etch position vs.** V_{th}**:** A weaker positive trend is observed. Higher etch position values correspond to shorter effective gate lengths, which increases short-channel effects and leads to a reduction in V_{th} (roll-off).
- T_{ox} **vs.** N_d**:** No noticeable correlation is observed (as expected from LHS sampling), confirming that the dataset is well-designed without confounding between input features.

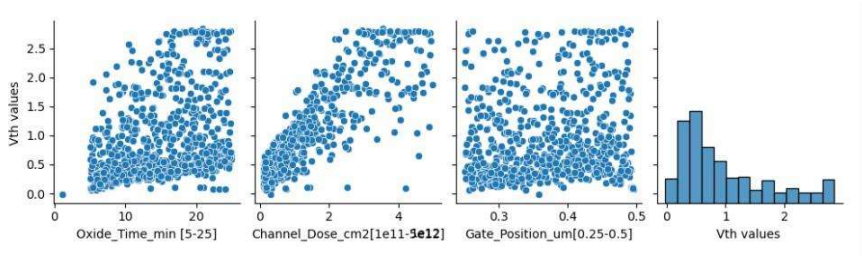


Figure 4.1: Pair plot of the 3-parameter TCAD dataset ($n = 686$) showing pairwise scatter plots and univariate KDE distributions for all features and target.

4.2.2 Model Performance Comparison

Table 4.2 summarizes the performance of all models evaluated on the 3-parameter dataset.

Table 4.2: 3-Parameter Model Performance Comparison (Post-Midsemester)

Model	R^2	RMSE (V)	Max Err (V)	Notes
Linear Regression	0.887	–	–	Underfits interactions
Poly Reg. (deg. 2)	0.889	–	–	Marginal improvement
Random Forest	0.905	–	–	500 trees, depth 10
Neural Network	0.9546	0.132	0.450	Tanh activations, Adam optimizer
XGBoost	**0.9666**	–	–	**Best model**; 300 trees, lr=0.05

4.2.3 Predicted vs. Actual: Random Forest

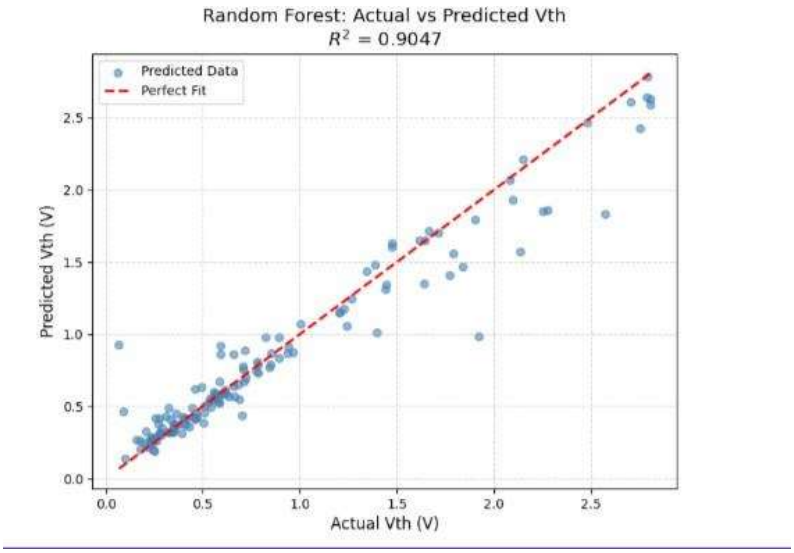


Figure 4.2: Predicted vs. actual V_{th} scatter plot for Random Forest on the held-out test set (R^2 = 0.9047). It can be seen from the plot that while for lower V_{th} the model performs well, it struggles at higher V_{th} as Random Forest is not good at capturing smooth trends.

4.2.4 Neural Network Learning Curve

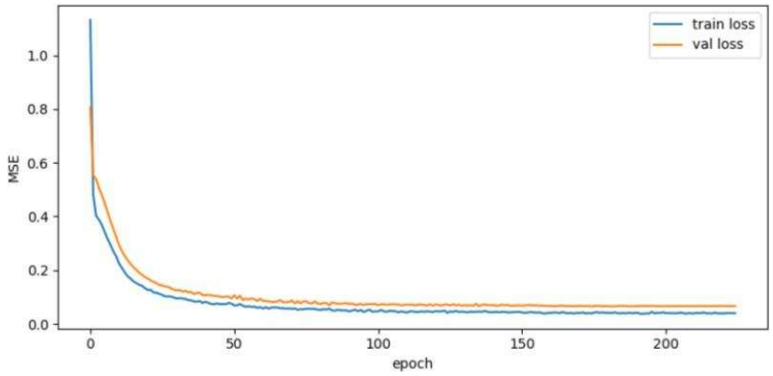


Figure 4.3: Training and validation loss curves for VthNet over training epochs. The close gap between train and validation loss indicates good generalization without overfitting.

4.2.5 Predicted vs. Actual: Neural Network

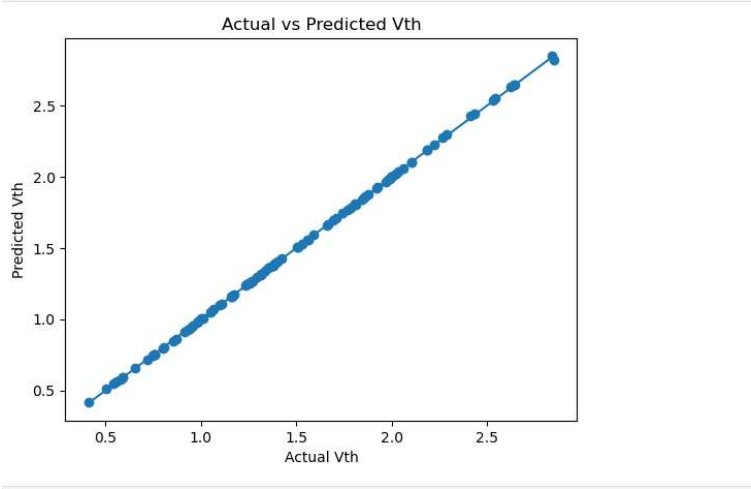


Figure 4.4: Predicted vs. actual V_{th} scatter plot for the Neural Network (VthNet) on the held-out test set ($R^2 = 0.9546$). Points close to the line indicates accurate predictions. Deviations at high V_{th} (> 1.8 V) tell us about the sparsity of high-dose samples.

4.2.6 Predicted vs. Actual: XGBoost

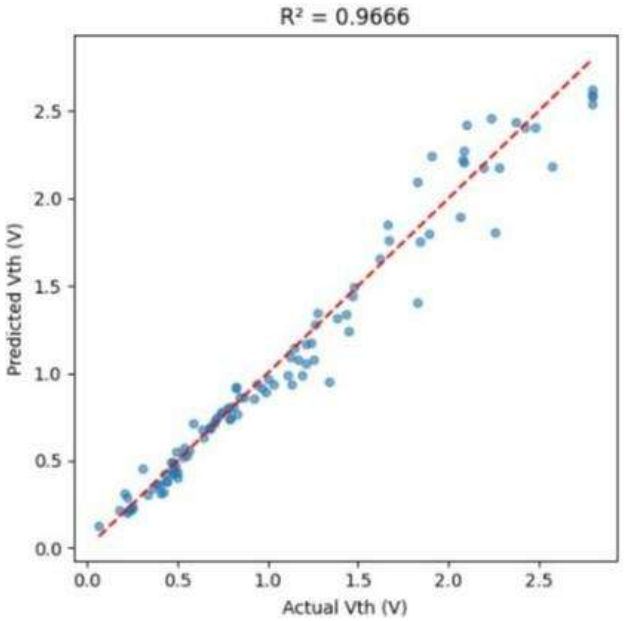


Figure 4.5: Predicted vs. actual V_{th} scatter plot for XGBoost on the held-out test set (R^2 = 0.9666). XGBoost achieves the tightest scatter of all models evaluated. This reflects the advantage of having sequential trees, where each tree corrects the residual error made by the previous one, thus acting as a correction over independent tree averaging (Random Forest) and smooth global function fitting (Neural Network). The characteristic tree-based discretization pattern is visible at extreme V_{th} values, representing the primary remaining limitation of this model class.

4.2.7 Web Interface and Digital Twin Deployment

To transfer the surrogate model from this experimental environament to an interactive two way web interface with the users we have developed a full stack web application for this model .The interface is a **digital twin** of the MOSFET fabrication process, which acts as a two way bridge between the predicting model and the user available athttps://website-demo-h607.onrender.com.

The application looks simple but its robust in nature:

- **Backend:** It is Built on **Spring Boot** framework, which handles the interface requests using the RESTful API .**Maven** manages the project dependencies and the building process is automated.
- **Server:** The App runs with embedded **Apache Tomcat** server, where requests are handled quickly and a separate server to work is not required.

- **Frontend:**A customized styling of the interface is done using **CSS** that is simple in syntax, dynamic, easily explainable.
- **Deployment:** The full stack application is hosted and deployed on **Render** cloud platform, allowing public access at https://website-demo-h607.onrender.com.

The interface supports two main workflows:

1. **Forward Prediction:** Users can change the sliders that edit the T_{ox}, N_d, The given values go into the backend which passes them to the trained ML model and gets the estimated V_{th}back to the interface shortly.
2. **Design Visualization:**The interface portrays how electrically the device functions and helps in understanding in real life it exactly functions with machine learning.

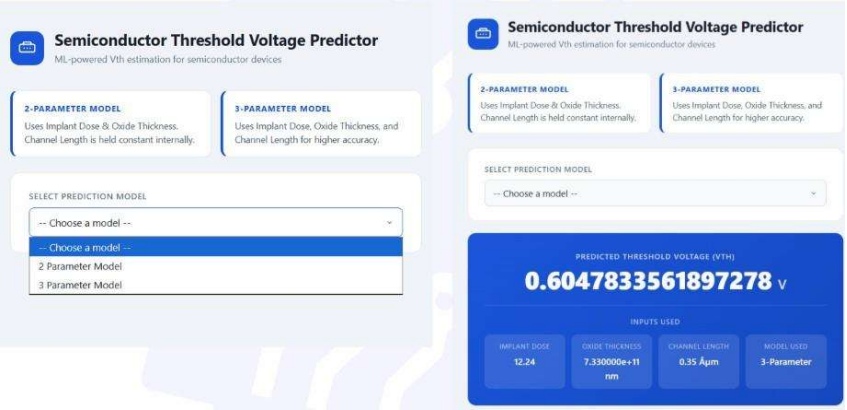


Figure 4.6: The Digital Twin web interface showing the parameter input dashboard (left) and the predicted V_{th} output display (right).

4.3 Discussion

XGBoost achieves the highest test $R^2 = 0.9666$ and outperforms the Neural Network by $\Delta R^2 \approx 0.012$ and Random Forest by $\Delta R^2 \approx 0.062$. This result is consistent with findings in the tabular data literature: gradient-boosted trees tend to outperform neural networks on small, structured datasets, where the sequential error-correction mechanism can concentrate modeling capacity on the hardest-to-predict samples. The Neural Network ranks second as it smoothly approximates the function. Both ensemble methods substantially outperform the polynomial and linear baselines, confirming the necessity of non-linear, interaction-aware models for this problem.

XGBoost's prediction scatter shows characteristic tree-based discretization: predictions cluster around the finite set of leaf values, creating a subtle staircase pattern particularly visible at extreme V_{th} values. This is an inherent limitation of tree-based models for

smooth regression functions. Nevertheless, the tighter overall scatter compared to both Random Forest and Neural Network confirms XGBoost as the best-performing model on this dataset.

The learning curve for VthNet shows healthy training dynamics: both train and validation loss decrease monotonically at the start, with the scheduler triggering learning rate halvings when progress stops. The validation loss (computed on dropout-disabled inference) is lower than training loss in many epochs—a classical signature of effective dropout regularization. Early stopping fires before the full 1000 epochs, preserving the best generalization checkpoint.

The scatter plot for VthNet confirms strong agreement across the entire V_{th} range. The maximum absolute error of 0.450 V occurs at the upper tail of the distribution (high N_d, thick oxide), where fewer training samples exist (the LHS design has some sparsity at extreme corners). This is a classical extrapolation challenge and motivates denser sampling in future data generation.

During the two-parameter baseline phase, a polynomial degree sweep was conducted to characterize the bias-variance tradeoff. The sweep confirms degree-2 as the optimal polynomial complexity. Degrees 3 and above show an increasing RMSE validation (over-fitting), despite further reductions in training error. This analysis justified the final polynomial model selection and also motivates the use of regularized models (XGBoost, NN) for the three-parameter case where the feature space is richer.

CHAPTER 5

Conclusion and Future Work

This work is an ML integrated work on optimal estimation of MOSFET's V_{th} , we extended this from a simple two parameter model to a more realistic 3 parameter model, the preliminary contributions are ass follows :

- A TCAD sample data simulated of quantity 686 done by the variation of the gate and oxide thickness and the implant dose and gate etch position utilizing the Silvaco Athena/Atlas.
- : Five models are used to fit the training data namely the linear regression, polynomial regression (degree 2), random forest, XGBoost, and an artificial neural network.
- **XGBoost achieved the best overall performance** ($R^2 = 0.9666$), outperforming the Neural Network VthNet ($R^2 = 0.9546$) by $\Delta R^2 \approx 0.012$ and Random Forest ($R^2 = 0.9047$) by $\Delta R^2 \approx 0.062$. This matches the exact expectation of the reality of the theory where the gradient boosted tress have edge over all the models as they can focus on harder samples by detecting and improving the errors over time.
- An interactive web based digital twin has been deployed for users to dynamically interact with underlying model to predict the Vth according to their requirement.

Future work will concentrate on further extending this framework. First, the model can be expanded to a seven-parameter MOSFET design space. This means that it can include things like the energy of the source/drain implant, the temperature of the anneal, the thickness of the spacer oxide, the doping of the substrate, and the dose of the halo implant. This makes the design much more complicated, so techniques like active learning and Bayesian optimization can be used to cut down on the number of TCAD simulations needed.

Adding the estimation of uncertainty (with Gaussian processes or Bayesian neural networks) would also make the predictions more accurate. Another way is to use sampling

that is aware of variability to better capture changes in the process. Lastly, checking the model against false device measurements would be a big step toward making it useful in the real world.

BIBLIOGRAPHY

[1] S. Woo, J. Kim, and H. Park, "Prediction of device characteristics of feedback FETs using TCAD-ML," *Micromachines*, vol. 14, no. 5, 2023.

[2] S. Choi, J. Lee, and B. Kim, "Automatic prediction of MOSFETs threshold voltage by machine learning," in *Proc. IEEE EDTM*, 2023.

[3] B. C. Wang, X. Zhang, and Y. Li, "A machine learning approach to TCAD model calibration for semiconductor devices," *Nuclear Science and Techniques*, vol. 34, 2023.

[4] P. Jungmann, M. Schroter, and T. Muller, "TCAD-enabled machine learning—an efficient workflow for dataset generation and preprocessing," *IEEE Trans. Semicond. Manuf.*, vol. 36, no. 2, pp. 201–210, 2023.

[5] Y. Lyu, Z. Chen, and W. Zhang, "Machine learning-assisted device modeling with process variations for nanosheet FETs," *J. Electron Devices Soc.*, vol. 11, pp. 453–462, 2023.

[6] M. Ji, H. Wang, and K. Park, "A stacked regression-based TCAD-ML framework for accelerated GaN transistor design," *Microelectronics Journal*, vol. 148, 2026.

[7] H. Zhang, L. Liu, and J. Chen, "ML-based device modeling and performance optimization for FinFETs," *IEEE Trans. Circuits Syst. II*, vol. 70, no. 8, pp. 2891–2895, 2023.

[8] K. M. Mamun et al., "A comprehensive review of machine learning approaches for semiconductor device modeling," *IEEE Access*, vol. 13, pp. 11024–11058, 2025.

[9] P. Raut et al., "A comprehensive review on next-generation modeling and optimization for semiconductor devices," *IEEE Access*, 2025.

CHAPTER A

TCAD Simulation Scripts

A.1 BUILDER: Silvaco Athena Process Script

Listing A.1: Silvaco Athena process simulation script (BUILDER).

```
go athena

# --- Mesh & Init ---
line x loc=0.0 spac=0.15
line x loc=0.6 spac=0.15
line y loc=0.0 spac=0.01
line y loc=0.8 spac=0.2
init orientation=100 c.phos=1e14 space.mul=2

# --- Process Steps ---
diffus time=30 temp=1000 dryo2 press=1.00 hcl=3
etch oxide thick=0.02
implant boron dose=8e12 energy=100 pearson
diffus temp=950 time=100 weto2 hcl=3
diffus time=50 temp=1000 t.rate=4.000 dryo2 press=0.10 hcl=3
diffus time=220 temp=1200 nitro press=1
diffus time=90 temp=1200 t.rate=-4.444 nitro press=1
etch oxide all
diffus time=20 temp=1000 dryo2 press=1 hcl=3
etch oxide all

# ================================================
# THE 3 ML KNOBS
# ================================================
# KNOB 1: Gate Oxide Thickness
diffus time=24.45471945 temp=925 dryo2 press=1.00

# KNOB 2: Channel Doping
implant boron dose=2.44E+11 energy=10 pearson
```

```
depo poly thick=0.4 divi=5

# KNOB 3: Gate Length
etch poly left p1.x=0.403284979

# --- Finalize Structure ---
method fermi compress
diffus time=4 temp=900 weto2 press=1.0
implant phosphor dose=3.0e13 energy=20 pearson
depo oxide thick=0.120 divi=4
etch oxide thick=0.120
implant arsenic dose=5.0e15 energy=50 pearson
method fermi compress
diffus time=1 temp=900 nitro press=1.0
etch oxide left p1.x=0.2
depo alumin thick=0.03 divi=2
etch alumin right p1.x=0.18
structure mirror right
electrode name=gate x=0.5 y=0.1
electrode name=source x=0.1
electrode name=drain x=1.1
electrode name=substrate backside
structure outfile=my_device.str
quit
```

A.2 TESTER: Silvaco Atlas Device Script

Listing A.2: Silvaco Atlas device simulation script (TESTER).

```
go atlas

mesh infile=my_device.str
models cvt srh print
contact name=gate n.poly
interface qf=3e10
method newton
solve init
solve vdrain=0.1
log outfile=iv.log
solve vgate=0 vstep=0.05 vfinal=3.0 name=gate
log off
extract                name    ="    FINAL_VTH    "              \
  (xintercept(maxslope(curve(abs(v."gate"),abs(i."drain")))) \
```

```
   - abs(ave(v."drain"))/2.0)
quit
```